



Snake Game

Programmation Orientée Objet

SOMMAIRE

I. Introduction	3
II. Snake Game.....	4
III. Intervenants	5
1. Les rôles.....	5
2. Le public ciblé.....	6
IV. Motivation du projet.....	7
1. Les aspects de notre motivation.....	7
2. Les difficultés.....	8
3. Les améliorations possibles	8
V. Procéder dans la création du projet	9
1. Outils utilisés	9
2. Bibliothèque utilisée.....	9
3. Contenu du programme	10
2. Les attributs de la classe.....	10
VI. Coût du projet.....	17
VII. Conclusion.....	18
VIII. Annexe.....	19
1. Le programme :	19

I. Introduction

Il fut un temps où les téléphones servaient qu'à passer des appels ou envoyer quelques messages. Mais il y avait un jeu dont on ne pouvait pas se passer qui était le fameux jeu du serpent.

Au départ, le jeu du serpent était un jeu qui se jouait en salle d'arcade. Il faudra attendre la fin des années 90 pour qu'un développeur chez Nokia, nommé Taneli Armento trouve l'idée de programmer une version de Snake sur le Nokia 6110. Puis, 3 ans plus tard, en l'an 2000, que la version que nous avons tous connue sur le Nokia 3310, fait son apparition et devient populaire, et qui marquera plusieurs générations.

Notre projet "Snake Game" a été réalisé dans le cadre d'un projet informatique à réaliser en cours de programmation orientée objet. Il s'agit d'un jeu rétro entièrement réalisé avec le langage de programmation Java.

Le choix du projet était assez évident car c'est un jeu qu'on affectionne particulièrement depuis très jeune. Nous voulions reproduire ce jeu pour nous permettre de comprendre le fonctionnement du jeu.

II. Snake Game

Notre projet s'intitule "Snake Game". Il consiste à programmer en Java du célèbre jeu du serpent qui doit manger des pommes pour grandir.

Le fonctionnement est simple. Le jeu se joue seul. Le joueur contrôle un carré vert représentant le serpent. Le but est de manger les petits carrés rouges, qui symbolisent les pommes, ce qui fait grandir le serpent au fur et à mesure. La partie se termine si le serpent touche un mur ou se mord lui-même.

III. Intervenants

1. Les rôles

Pour la réalisation de notre projet, chacune des personnes de ce groupe a joué un rôle à titrer d'un métier : le chef de projet, le concepteur, le développeur Java, l'intégrateur graphique, le testeur et le documentaliste.

Pour les responsabilités, on a :

Le chef de projet qui planifie les étapes de réalisation, définit les priorités et de cibler les tâches

Le concepteur qui étudie les logiques du jeu, définit les classes, leurs attributs et les interactions entre les différents objets.

Le développeur Java qui écrit le code source, implémente les fonctionnalités principales c'est à dire les mouvements, les collisions, l'interface.

L'intégrateur graphique qui met en place l'interface utilisateur avec la fenêtre, les boutons et l'affichage graphique du jeu.

Le testeur vérifie que le programme fonctionne correctement, corrige les erreurs et s'assure du bon fonctionnement global du programme.

Le documentaliste s'occupe de rédiger la documentation du projet avec le rapport et prépare le diaporama.

2. Le public ciblé

Le jeu s'adresse principalement :

- Au grand public qui souhaite simplement jouer à un jeu retro simple d'accès.
- Aux étudiants en informatique qui veulent comprendre les bases de la programmation orienter objet à travers un cas concret.
- Aux enseignants qui peuvent s'en servir comme support pédagogique pour introduire les concepts de classes, d'évènement et de boucle.

IV. Motivation du projet

1. Les aspects de notre motivation

Nous avons voulu faire ce projet de création du jeu du serpent à notre niveau pour plusieurs raisons.

Pour l'aspect pédagogique, nous avons trouvé que ce jeu prenait en compte plusieurs choses que nous avons appris à faire lors de notre semestre en cours de programmation orientée objet.

Pour l'aspect technique, nous avons beaucoup aimé apprendre à manipuler une librairie swing pour créer une interface graphique interactive.

Pour l'aspect personnel et créatif, nous avons souhaité relever le défi de refaire le jeu auquel on a beaucoup joué auparavant et le refaire à notre manière avec les touches, le score et les messages.

2. Les difficultés

Nous avons rencontré de nombreuses difficultés, au cours de notre travail, tels que :

- Pour l'utilisation de la librairie swing car nous ne l'avons étudié en classe.
- Pour le testeur qui a dû trouver toutes les petites erreurs et permettre de faire en sorte que le programme fonctionne correctement.
- Pour faire le lien avec le clavier, des mouvements pouvait être bloquer ou ne plus fonctionner à certain moment.
- Pour finir la conception du projet en temps et en heure, demandé par le chef de projet.

Ces difficultés ont été surmonté grâce à des tests réguliers, l'analyse pas à pas du code et l'utilisation de l'aide de VSCode pour déboguer visuellement.

3. Les améliorations possibles

Les améliorations possibles qui pourrait rendre le jeu plus complet serait :

- L'ajout d'un menu principal avec des options de vitesse, de taille de grille et d'un chronomètre.
- L'intégration du meilleur score pour permettre au joueur de se dépasser et d'avoir le meilleur score.
- L'ajout d'effet sonore et musique.
- L'adaptation du jeu au format mobile.
- L'ajout notamment de différentes personnalisations du serpent, de l'arrière-plan et des « pommes ».

V. Procéder dans la création du projet

1. Outils utilisés

Pour réaliser le projet, nous avons utilisé certains outils, qui sont :

- Le langage de programmation Java
- L'environnement de développement intégrés (IDE) VSCode
- Un Kit de Développement Java (JDK)

2. Bibliothèque utilisée

Nous avons utilisé différentes librairies pour la création de notre programme :

- `javax.swing` : pour la fenêtre, les boutons et le dessin du panneau de jeu.
- `java.awt` : pour le dessin graphique et la gestion des couleurs.
- `java.util` : pour les structures de données (`ArrayList`, `Random`) et le timer (`javax.swing.Timer`).

3. Contenu du programme

1. Classe principale

Notre programme n'a qu'une seule classe, elle représente l'ensemble du jeu avec l'affichage, la logique et l'interaction du clavier.

Dans cette classe nous avons :

- L'héritage de JPanel.
- L'implémentation des interfaces :
 - ActionListener qui réagit au timer
 - KeyListener qui sert à la gestion du clavier

2. Les attributs de la classe

a. Les attributs constants :

```
private static final int BOX = 25;  
private static final int WIDTH = 800;  
private static final int HEIGHT = 600;
```

Ce sont des variables finales, leur valeur ne change jamais.
Elles définissent la taille du jeu.

b. Les attributs liés au serpent :

```
private final ArrayList<Point> snake = new ArrayList<>();  
private String direction = "RIGHT";
```

- **Snake** est la liste des positions (chaque segment est un Point)
- **Direction** est la direction actuelle de la tête du serpent

c. Les attributs liés au gameplay :

```
private Point food;  
private int score = 0;  
private final Timer timer;
```

- Food est la position de la pomme
- Score est le score du joueur
- Timer, c'est ce qui déclenche l'action du jeu toutes les 100 ms

Le **Timer** appelle automatiquement `actionPerformed()` qui est la logique du jeu.

3. Les méthodes

a. La méthode du constructeur :

```
public SnakeGame() { ... }
```

Elle initialise :

- La taille du panneau avec `setPreferredSize(new Dimension(WIDTH, HEIGHT));`
- La couleur avec `setBackground(Color.BLACK);`
- Les écouteurs de clavier avec `setFocusable(true);` et `addKeyListener(this);`
- Le serpent avec `snake.add(new Point(9 * BOX, 10 * BOX));`
- La première pomme avec `spawnFood();`
- Le timer avec `timer = new Timer(100, this);`

C'est le rôle typique d'un constructeur c'est à dire initialiser **l'objet**.

```
private void move()
```

Gère le déplacement du serpent :

- Calcule la nouvelle position de la tête
- Vérifie si la pomme est mangée
- Allonge ou fait avancer le serpent

C'est la **méthode centrale de la logique**.

```
private void checkCollision()
```

Vérifie les collisions :

- Des murs
- Du corps du serpent

Appelle gameOver() si nécessaire.

```
private void gameOver()
```

Arrête le timer et affiche le message “game over” avec le score.

b. La méthode de la logique du jeu

```
private void spawnFood()
```

Génère une nouvelle pomme à une position aléatoire.

c. La méthode d’affichage avec l’hérités de Swing

```
protected void paintComponent
```

Redessine :

- Le serpent
- La pomme
- Le score

C’est la méthode classique de dessin dans Swing.

d. La méthode du timer avec ActionListener

```
public void actionPerformed
```

Le timer appel automatiquement toutes les 100ms move(), checkCollision() et repaint()

C'est la boucle d'animation du jeu

e. La méthode de clavier avec KeyListener

Public void keyPressed(KeyEvent e)

Change la direction selon la touche appuyée.

Cependant 2 règles importantes, qui sont :

- Pas de demi-tour direct
- 4 flèches = 4 directions

public void startGame()

Réinitialise tout et relance le timer.

Utilisée en cliquant sur le bouton "Démarrer".

f. La méthode main du jeu

Le rôle du main est de créer la fenêtre, ajouter les boutons et ajouter le panneau de jeu.

On a :

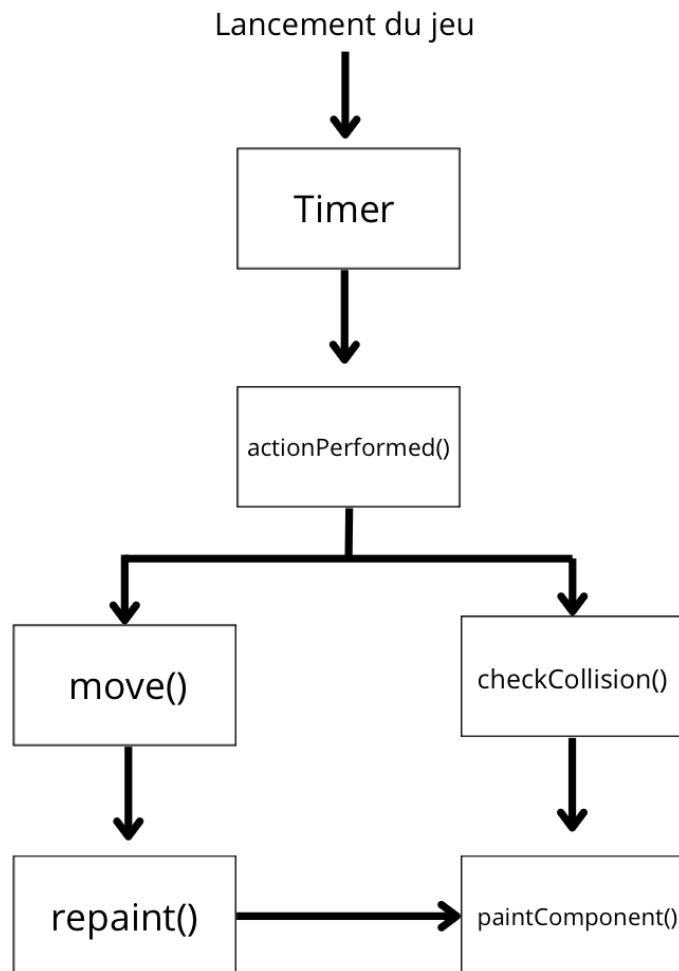
- Une JFrame
- Un JPanel pour les boutons
- Un objet SnakeGame placé au centre

Ici, nous voyons la relation entre classes Swing et ta classe SnakeGame :

JFrame

- JPanel (boutons)
- SnakeGame (JPanel du jeu)

4. Schéma d'ordonnance



VI. Coût du projet

Pour réaliser ce projet, nous avons mis environ deux mois du début à la fin. Nous nous sommes vite rendu compte que toutes les parties ne demander pas le même investissement, certaines étaient simples, d'autres beaucoup plus difficiles.

La partie qui nous a pris le plus de temps, c'est tout ce qui touche au déplacement du serpent, donc la méthode `move()`. Nous avons dû réfléchir à comment calculer la nouvelle position de la tête, comment faire grandir le serpent quand il mange une pomme, et comment mettre à jour correctement la liste du corps. C'est là que nous avons passé le plus de temps à tester, corriger et retester, parce que le moindre détail pouvait provoquer un bug comme le serpent qui se téléporte, qui saute une case, qui grandit sans raison... Ça nous a demandé plus d'attention que le reste, facilement trois fois plus de travail que d'autres fonctionnalités.

La vérification des collisions (`checkCollision()`) nous a pris du temps aussi, cependant cela était un peu plus simple. Nous savions que nous devions vérifier les collisions avec les murs et le corps du serpent. Ainsi, même si ça demandait de la rigueur, cela était moins complexe que le déplacement.

Pour ce qui est de l'affichage (`paintComponent()`), cela a été beaucoup plus rapide. Une fois que la logique du jeu fonctionnait bien, dessiner le serpent, la pomme et le score, cela s'est fait plutôt facilement. La partie affichage a pris moins de temps, car elle dépendait surtout de choses déjà en place.

Enfin, les choses les plus rapides ont été la mise en place des boutons (Démarrer, Quitter) et la configuration de la fenêtre dans le main. Cela était surtout du placement visuel. Rien de très compliqué à ce niveau-là.

Finalement, le temps que nous avons investi n'a pas du tout été réparti équitablement. Certaines parties demandaient un peu d'organisation, alors que d'autres nécessitaient de la logique, des essais et beaucoup de corrections.

VII. Conclusion :

En conclusion, ce projet nous a permis de mieux comprendre les principaux fondements de la Programmation Orientée Objet à travers un cas concret. Même si notre jeu repose principalement sur une seule classe, nous avons pu voir comment organiser un programme avec un constructeur, des attributs, plusieurs méthodes, ainsi que l'héritage et l'utilisation d'interfaces comme ActionListener et KeyListener. Le fait d'appliquer ces notions dans un jeu comme Snake nous a vraiment aidés à les assimiler plus facilement.

Nous avons aussi réalisé qu'avant de commencer à coder, il est important de réfléchir à la structure du programme et au rôle de chaque partie : l'affichage, la logique du serpent, la gestion du clavier, le timer... Cette étape nous a permis d'éviter beaucoup d'erreurs et de mieux comprendre comment tout fonctionne ensemble.

Enfin, ce travail nous a également aidé à comprendre qu'une bonne conception et un bon travail d'équipe vaut mieux qu'une implémentation directe et à progresser sur le plan technique et méthodologique de la programmation. Il nous a montré que la conception et la communication sont essentielles pour obtenir un résultat propre et fonctionnel.

VIII. Annexe :

1. Le programme :

```
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.util.Random;
import javax.swing.*;

public class SnakeGame extends JPanel implements ActionListener, KeyListener {

    private static final int BOX = 25;
    private static final int WIDTH = 800;
    private static final int HEIGHT = 600;

    private final ArrayList<Point> snake = new ArrayList<>();
    private String direction = "RIGHT";
    private Point food;
    private int score = 0;

    private final Timer timer;

    public SnakeGame() {
        setPreferredSize(new Dimension(WIDTH, HEIGHT));
        setBackground(Color.BLACK);
        setFocusable(true);
        addKeyListener(this);

        snake.add(new Point(9 * BOX, 10 * BOX));
        spawnFood();

        timer = new Timer(100, this);
    }

    private void spawnFood() {
        Random rand = new Random();
        int x = rand.nextInt(WIDTH / BOX) * BOX;
        int y = rand.nextInt(HEIGHT / BOX) * BOX;
        food = new Point(x, y);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        for (int i = 0; i < snake.size(); i++) {
            if (i == 0) g.setColor(Color.GREEN);
        }
    }
}
```

```

else g.setColor(new Color(0, 150, 0));
g.fillRect(snake.get(i).x, snake.get(i).y, BOX, BOX);
}

g.setColor(Color.RED);
g.fillRect(food.x, food.y, BOX, BOX);

g.setColor(Color.WHITE);
g.drawString("Score: " + score, 10, 50);
}

@Override
public void actionPerformed(ActionEvent e) {
move();
checkCollision();
repaint();
}

private void move() {
Point head = snake.get(0);
int x = head.x;
int y = head.y;

switch (direction) {
case "LEFT" -> x -= BOX;
case "UP" -> y -= BOX;
case "RIGHT" -> x += BOX;
case "DOWN" -> y += BOX;
}

Point newHead = new Point(x, y);

if (newHead.equals(food)) {
score++;
spawnFood();
} else {
snake.remove(snake.size() - 1);
}

snake.add(0, newHead);
}

private void checkCollision() {
Point head = snake.get(0);

if (head.x < 0 || head.y < 0 || head.x >= WIDTH || head.y >= HEIGHT) {
gameOver();
}

for (int i = 1; i < snake.size(); i++) {
if (head.equals(snake.get(i))) {
gameOver();
}
}
}

```

```

break;
}
}
}

private void gameOver() {
timer.stop();
JOptionPane.showMessageDialog(this, "Game Over! Score : " + score);
}

@Override
public void keyPressed(KeyEvent e) {
int key = e.getKeyCode();

if (key == KeyEvent.VK_LEFT && !direction.equals("RIGHT")) direction = "LEFT";
else if (key == KeyEvent.VK_UP && !direction.equals("DOWN")) direction = "UP";
else if (key == KeyEvent.VK_RIGHT && !direction.equals("LEFT")) direction = "RIGHT";
else if (key == KeyEvent.VK_DOWN && !direction.equals("UP")) direction = "DOWN";
}

@Override
public void keyReleased(KeyEvent e) {}

@Override
public void keyTyped(KeyEvent e) {}

public void startGame() {
score = 0;
direction = "RIGHT";
snake.clear();
snake.add(new Point(9 * BOX, 10 * BOX));
spawnFood();
timer.start();
}

public static void main(String[] args) {
JFrame frame = new JFrame("Snake Game");
SnakeGame gamePanel = new SnakeGame();

JPanel buttonPanel = new JPanel(new FlowLayout());
buttonPanel.setOpaque(false);

JButton startButton = new JButton("Démarrer");
JButton quitButton = new JButton("Quitter");

startButton.setFocusable(false);
quitButton.setFocusable(false);

startButton.addActionListener(e -> gamePanel.startGame());
quitButton.addActionListener(e -> System.exit(0));

buttonPanel.add(startButton);

```

```
buttonPanel.add(quitButton);

frame.setLayout(new BorderLayout());
frame.add(buttonPanel, BorderLayout.NORTH);
frame.add(gamePanel, BorderLayout.CENTER);

frame.pack();
frame.setLocationRelativeTo(null);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
```